

Exercise 1: Control Structures

Scenario 1: Apply 1% discount on loan interest for customers above 60 years

Program:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_customers IS
        SELECT CustomerID, Name, DOB FROM Customers;
    v_age NUMBER;
BEGIN
    FOR cust IN c_customers LOOP
        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, cust.DOB) / 12);
        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = cust.CustomerID;
            DBMS_OUTPUT.PUT_LINE('Discount applied for ' || cust.Name ||
                                ' (age ' || v_age || ')');
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

Output:

```
Discount applied for Ramesh Kumar (age 65)
Discount applied for Susan Wilson (age 72)
```

PL/SQL procedure successfully completed.

Scenario 2: Set IsVIP flag to TRUE for customers with balance over \$10,000

Program:

```
SET SERVEROUTPUT ON;

-- add the IsVIP column first (only needed once)
ALTER TABLE Customers ADD IsVIP CHAR(1) DEFAULT 'N';

DECLARE
    CURSOR c_cust IS
        SELECT CustomerID, Name, Balance FROM Customers;
BEGIN
    FOR c IN c_cust LOOP
        IF c.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'Y'
            WHERE CustomerID = c.CustomerID;
            DBMS_OUTPUT.PUT_LINE(c.Name || ' promoted to VIP status.');
```

```
END;  
/
```

Output:

Table altered.

Meena Iyer promoted to VIP status.
Rahul Verma promoted to VIP status.

PL/SQL procedure successfully completed.

Scenario 3: Send reminder for loans due in the next 30 days

Program:

```
SET SERVEROUTPUT ON;  
  
DECLARE  
    CURSOR c_loans IS  
        SELECT L.LoanID, L.EndDate, C.Name  
        FROM Loans L  
        JOIN Customers C ON L.CustomerID = C.CustomerID  
        WHERE L.EndDate BETWEEN SYSDATE AND SYSDATE + 30;  
BEGIN  
    FOR l IN c_loans LOOP  
        DBMS_OUTPUT.PUT_LINE('Reminder: Dear ' || l.Name ||  
                               ', your loan #' || l.LoanID ||  
                               ' is due on ' ||  
                               TO_CHAR(l.EndDate, 'DD-MON-YYYY'));  
    END LOOP;  
END;  
/
```

Output:

Reminder: Dear John Doe, your loan #12 is due on 15-JUL-2026
Reminder: Dear Jane Smith, your loan #14 is due on 22-JUL-2026

PL/SQL procedure successfully completed.

Exercise 2: Error Handling

Scenario 1: SafeTransferFunds - transfer money with rollback on error

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_from_account IN NUMBER,
    p_to_account   IN NUMBER,
    p_amount       IN NUMBER
) AS
    v_balance NUMBER;
    insufficient_funds EXCEPTION;
BEGIN
    -- lock the source account row
    SELECT Balance INTO v_balance FROM Accounts
    WHERE AccountID = p_from_account
    FOR UPDATE;

    IF v_balance < p_amount THEN
        RAISE insufficient_funds;
    END IF;

    UPDATE Accounts SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;
    UPDATE Accounts SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer of Rs.' || p_amount || ' completed
successfully.');
```

EXCEPTION

```
    WHEN insufficient_funds THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Insufficient funds in account ' ||
p_from_account);
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Account not found.');
```

WHEN OTHERS THEN

```
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

-- test the procedure
BEGIN
    SafeTransferFunds(1, 2, 500);           -- valid transfer
    SafeTransferFunds(1, 2, 999999);       -- insufficient funds
END;
/
```

Output:

Procedure created.

Transfer of Rs.500 completed successfully.
Error: Insufficient funds in account 1

PL/SQL procedure successfully completed.

Scenario 2: UpdateSalary - increase salary by percentage, handle missing employee

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_emp_id  IN NUMBER,
    p_percent IN NUMBER
) AS
    v_old_salary NUMBER;
    v_new_salary NUMBER;
BEGIN
    SELECT Salary INTO v_old_salary FROM Employees
        WHERE EmployeeID = p_emp_id;

    v_new_salary := v_old_salary + (v_old_salary * p_percent / 100);

    UPDATE Employees SET Salary = v_new_salary
        WHERE EmployeeID = p_emp_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Salary updated for Employee ' || p_emp_id ||
        ' from Rs.' || v_old_salary ||
        ' to Rs.' || v_new_salary);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Employee with ID ' || p_emp_id || ' does
not exist. ');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

BEGIN
    UpdateSalary(1, 10);      -- existing employee
    UpdateSalary(99, 5);     -- non-existing employee
END;
/
```

Output:

Procedure created.

Salary updated for Employee 1 from Rs.70000 to Rs.77000
Error: Employee with ID 99 does not exist.

PL/SQL procedure successfully completed.

Scenario 3: AddNewCustomer - insert customer, handle duplicate ID

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_cust_id IN NUMBER,
    p_name     IN VARCHAR2,
    p_dob      IN DATE,
    p_balance  IN NUMBER
) AS
BEGIN
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
    VALUES (p_cust_id, p_name, p_dob, p_balance, SYSDATE);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Customer ' || p_name || ' added successfully.');
```

EXCEPTION

```
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Error: Customer with ID ' || p_cust_id ||
                               ' already exists.');
```

WHEN OTHERS THEN

```
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

BEGIN
    AddNewCustomer(3, 'Ravi Prakash',
        TO_DATE('1988-04-12', 'YYYY-MM-DD'), 5000);
    -- try inserting an already existing customer
    AddNewCustomer(1, 'Duplicate John',
        TO_DATE('1985-05-15', 'YYYY-MM-DD'), 2000);
END;
/
```

Output:

Procedure created.

Customer Ravi Prakash added successfully.
Error: Customer with ID 1 already exists.

PL/SQL procedure successfully completed.

Exercise 3: Stored Procedures

Scenario 1: ProcessMonthlyInterest - apply 1% interest to all savings accounts

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
    v_count NUMBER;
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    v_count := SQL%ROWCOUNT;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Monthly interest applied to ' || v_count ||
                          ' savings accounts.');
```

END;

/

```
BEGIN
    ProcessMonthlyInterest;
END;
```

/

Output:

Procedure created.

Monthly interest applied to 3 savings accounts.

PL/SQL procedure successfully completed.

Scenario 2: UpdateEmployeeBonus - give bonus % to employees in a department

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department IN VARCHAR2,
    p_percent     IN NUMBER
) AS
    v_count NUMBER;
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_percent / 100)
    WHERE Department = p_department;

    v_count := SQL%ROWCOUNT;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE(v_count || ' employees in ' || p_department ||
                          ' got a ' || p_percent || '% bonus.');
```

END;

```

/

BEGIN
    UpdateEmployeeBonus('IT', 15);
    UpdateEmployeeBonus('HR', 10);
END;
/

```

Output:

Procedure created.

```

1 employees in IT got a 15% bonus.
1 employees in HR got a 10% bonus.

```

PL/SQL procedure successfully completed.

Scenario 3: TransferFunds - transfer with balance check

Program:

```

SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_acc IN NUMBER,
    p_to_acc   IN NUMBER,
    p_amount   IN NUMBER
) AS
    v_bal NUMBER;
BEGIN
    SELECT Balance INTO v_bal FROM Accounts
        WHERE AccountID = p_from_acc;

    IF v_bal < p_amount THEN
        DBMS_OUTPUT.PUT_LINE('Transfer failed: Insufficient balance in
account ' ||
                                p_from_acc);
        RETURN;
    END IF;

    UPDATE Accounts SET Balance = Balance - p_amount
        WHERE AccountID = p_from_acc;
    UPDATE Accounts SET Balance = Balance + p_amount
        WHERE AccountID = p_to_acc;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transferred Rs.' || p_amount ||
        ' from account ' || p_from_acc ||
        ' to account ' || p_to_acc);
END;
/

BEGIN
    TransferFunds(1, 2, 300);
    TransferFunds(1, 2, 50000);  -- insufficient balance
END;

```

/

Output:

Procedure created.

Transferred Rs.300 from account 1 to account 2

Transfer failed: Insufficient balance in account 1

PL/SQL procedure successfully completed.

Exercise 4: Functions

Scenario 1: CalculateAge - return age in years from DOB

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION CalculateAge(p_dob IN DATE)
RETURN NUMBER
IS
    v_age NUMBER;
BEGIN
    v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
    RETURN v_age;
END;
/

BEGIN
    DBMS_OUTPUT.PUT_LINE('Age of John Doe : ' ||
        CalculateAge(TO_DATE('1985-05-15', 'YYYY-MM-DD')));
    DBMS_OUTPUT.PUT_LINE('Age of Jane Smith : ' ||
        CalculateAge(TO_DATE('1990-07-20', 'YYYY-MM-DD')));
END;
/
```

Output:

Function created.

Age of John Doe : 41
Age of Jane Smith : 35

PL/SQL procedure successfully completed.

Scenario 2: CalculateMonthlyInstallment - EMI for a loan

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
    p_loan_amount    IN NUMBER,
    p_interest_rate  IN NUMBER,      -- annual %
    p_years          IN NUMBER
) RETURN NUMBER
IS
    v_monthly_rate NUMBER;
    v_months        NUMBER;
    v_emi           NUMBER;
BEGIN
    v_monthly_rate := (p_interest_rate / 100) / 12;
    v_months := p_years * 12;

    -- EMI formula: P * r * (1+r)^n / ((1+r)^n - 1)
    v_emi := p_loan_amount * v_monthly_rate *
        POWER(1 + v_monthly_rate, v_months) /
```

```

        (POWER(1 + v_monthly_rate, v_months) - 1);

    RETURN ROUND(v_emi, 2);
END;
/

BEGIN
    DBMS_OUTPUT.PUT_LINE('EMI for Rs.500000 at 8% for 5 years : Rs.' ||
        CalculateMonthlyInstallment(500000, 8, 5));
    DBMS_OUTPUT.PUT_LINE('EMI for Rs.200000 at 10% for 3 years : Rs.' ||
        CalculateMonthlyInstallment(200000, 10, 3));
END;
/

```

Output:

Function created.

```

EMI for Rs.500000 at 8% for 5 years : Rs.10138.2
EMI for Rs.200000 at 10% for 3 years : Rs.6453.44

```

PL/SQL procedure successfully completed.

Scenario 3: HasSufficientBalance - boolean check on an account

Program:

```

SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION HasSufficientBalance(
    p_account_id IN NUMBER,
    p_amount      IN NUMBER
) RETURN BOOLEAN
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance FROM Accounts
        WHERE AccountID = p_account_id;

    IF v_balance >= p_amount THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/

BEGIN
    IF HasSufficientBalance(1, 500) THEN
        DBMS_OUTPUT.PUT_LINE('Account 1 has sufficient balance for Rs.500');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Account 1 does NOT have sufficient balance for
Rs.500');
    END IF;

```

```
        IF HasSufficientBalance(1, 50000) THEN
            DBMS_OUTPUT.PUT_LINE('Account 1 has sufficient balance for
Rs.50000');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Account 1 does NOT have sufficient balance for
Rs.50000');
        END IF;
    END;
/
```

Output:

Function created.

Account 1 has sufficient balance for Rs.500

Account 1 does NOT have sufficient balance for Rs.50000

PL/SQL procedure successfully completed.

Exercise 5: Triggers

Scenario 1: UpdateCustomerLastModified - auto update LastModified on customer update

Program:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/

-- test the trigger
UPDATE Customers SET Balance = 2000 WHERE CustomerID = 1;
COMMIT;

SELECT CustomerID, Name, Balance,
       TO_CHAR(LastModified, 'DD-MON-YYYY HH24:MI:SS') AS LastModified
FROM Customers WHERE CustomerID = 1;
```

Output:

Trigger created.

1 row updated.

Commit complete.

CUSTOMERID	NAME	BALANCE	LASTMODIFIED
1	John Doe	2000	01-JUL-2026 10:24:15

Scenario 2: LogTransaction - log every new transaction into AuditLog

Program:

```
SET SERVEROUTPUT ON;

-- create the audit table first
CREATE TABLE AuditLog (
    LogID          NUMBER PRIMARY KEY,
    TransactionID  NUMBER,
    LogMessage     VARCHAR2(200),
    LogDate        DATE
);

CREATE SEQUENCE audit_log_seq START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
```

```

INSERT INTO AuditLog (LogID, TransactionID, LogMessage, LogDate)
VALUES (audit_log_seq.NEXTVAL, :NEW.TransactionID,
       'Transaction ' || :NEW.TransactionType || ' of Rs.' ||
       :NEW.Amount || ' on account ' || :NEW.AccountID,
       SYSDATE);
END;
/

-- test
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate,
                        Amount, TransactionType)
VALUES (3, 1, SYSDATE, 500, 'Deposit');
COMMIT;

```

```
SELECT LogID, TransactionID, LogMessage FROM AuditLog;
```

Output:

Table created.

Sequence created.

Trigger created.

1 row created.

Commit complete.

LOGID	TRANSACTIONID	LOGMESSAGE
1	3	Transaction Deposit of Rs.500 on account 1

Scenario 3: CheckTransactionRules - validate deposits and withdrawals

Program:

```

SET SERVEROUTPUT ON;

CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    v_balance NUMBER;
BEGIN
    -- deposits must be positive
    IF :NEW.TransactionType = 'Deposit' AND :NEW.Amount <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Deposit amount must be positive.');
```

END IF;

```

    -- withdrawals must not exceed balance
    IF :NEW.TransactionType = 'Withdrawal' THEN
        SELECT Balance INTO v_balance FROM Accounts
        WHERE AccountID = :NEW.AccountID;
        IF v_balance < :NEW.Amount THEN
            RAISE_APPLICATION_ERROR(-20002,
```

```

        'Withdrawal amount exceeds account balance.');
```

END IF;
 END IF;
END;
/

-- test with an invalid deposit
BEGIN
 INSERT INTO Transactions VALUES (4, 1, SYSDATE, -100, 'Deposit');
EXCEPTION
 WHEN OTHERS THEN
 DBMS_OUTPUT.PUT_LINE('Blocked: ' || SQLERRM);
END;
/

-- test with an invalid withdrawal
BEGIN
 INSERT INTO Transactions VALUES (5, 1, SYSDATE, 99999, 'Withdrawal');
EXCEPTION
 WHEN OTHERS THEN
 DBMS_OUTPUT.PUT_LINE('Blocked: ' || SQLERRM);
END;
/

Output:

Trigger created.

Blocked: ORA-20001: Deposit amount must be positive.

Blocked: ORA-20002: Withdrawal amount exceeds account balance.

PL/SQL procedure successfully completed.

Exercise 6: Cursors

Scenario 1: GenerateMonthlyStatements - print statements for the current month

Program:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_stmt IS
        SELECT C.CustomerID, C.Name,
               T.TransactionID, T.TransactionType,
               T.Amount, T.TransactionDate
        FROM Customers C
        JOIN Accounts A ON C.CustomerID = A.CustomerID
        JOIN Transactions T ON A.AccountID = T.AccountID
        WHERE T.TransactionDate BETWEEN TRUNC(SYSDATE, 'MM')
                                AND LAST_DAY(SYSDATE)
        ORDER BY C.CustomerID, T.TransactionDate;

    v_last_cust NUMBER := -1;
BEGIN
    FOR r IN c_stmt LOOP
        IF r.CustomerID <> v_last_cust THEN
            DBMS_OUTPUT.PUT_LINE('-----');
            DBMS_OUTPUT.PUT_LINE('Statement for ' || r.Name);
            v_last_cust := r.CustomerID;
        END IF;
        DBMS_OUTPUT.PUT_LINE('  ' ||
                              TO_CHAR(r.TransactionDate, 'DD-MON') || ' | ' ||
                              r.TransactionType || ' | Rs.' || r.Amount);
    END LOOP;
END;
/
```

Output:

```
-----
Statement for John Doe
  01-JUL | Deposit | Rs.200
  15-JUL | Deposit | Rs.500
-----
Statement for Jane Smith
  05-JUL | Withdrawal | Rs.300
```

PL/SQL procedure successfully completed.

Scenario 2: ApplyAnnualFee - deduct annual fee from all accounts

Program:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_acc IS
        SELECT AccountID, Balance FROM Accounts
        FOR UPDATE;
```

```

        v_fee NUMBER := 500;
BEGIN
    FOR a IN c_acc LOOP
        UPDATE Accounts
        SET Balance = Balance - v_fee
        WHERE CURRENT OF c_acc;

        DBMS_OUTPUT.PUT_LINE('Fee of Rs.' || v_fee ||
                              ' deducted from account ' || a.AccountID ||
                              '. New balance: Rs.' || (a.Balance - v_fee));
    END LOOP;
    COMMIT;
END;
/

```

Output:

Fee of Rs.500 deducted from account 1. New balance: Rs.500
 Fee of Rs.500 deducted from account 2. New balance: Rs.1000

PL/SQL procedure successfully completed.

Scenario 3: UpdateLoanInterestRates - adjust interest rates as per new policy

Program:

```

SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_loans IS
        SELECT LoanID, LoanAmount, InterestRate FROM Loans
        FOR UPDATE;
    v_new_rate NUMBER;
BEGIN
    FOR l IN c_loans LOOP
        -- new policy:
        -- loans above 100000 -> 0.5% discount
        -- otherwise -> 0.5% hike
        IF l.LoanAmount > 100000 THEN
            v_new_rate := l.InterestRate - 0.5;
        ELSE
            v_new_rate := l.InterestRate + 0.5;
        END IF;

        UPDATE Loans SET InterestRate = v_new_rate
        WHERE CURRENT OF c_loans;

        DBMS_OUTPUT.PUT_LINE('Loan ' || l.LoanID ||
                              ' updated from ' || l.InterestRate ||
                              '% to ' || v_new_rate || '%');
    END LOOP;
    COMMIT;
END;
/

```

Output:

Loan 1 updated from 5% to 5.5%

PL/SQL procedure successfully completed.

Exercise 7: Packages

Scenario 1: CustomerManagement - package for customer operations

Program:

```
SET SERVEROUTPUT ON;

-- package specification
CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddCustomer(p_id    IN NUMBER,
                          p_name  IN VARCHAR2,
                          p_dob   IN DATE,
                          p_bal   IN NUMBER);

    PROCEDURE UpdateCustomer(p_id    IN NUMBER,
                             p_name  IN VARCHAR2,
                             p_bal   IN NUMBER);

    FUNCTION GetCustomerBalance(p_id IN NUMBER) RETURN NUMBER;
END CustomerManagement;
/

-- package body
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer(p_id    IN NUMBER,
                          p_name  IN VARCHAR2,
                          p_dob   IN DATE,
                          p_bal   IN NUMBER) IS

    BEGIN
        INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
        VALUES (p_id, p_name, p_dob, p_bal, SYSDATE);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Customer ' || p_name || ' added.');
```

```
END;
```

```
    PROCEDURE UpdateCustomer(p_id    IN NUMBER,
                             p_name  IN VARCHAR2,
                             p_bal   IN NUMBER) IS

    BEGIN
        UPDATE Customers
        SET Name = p_name, Balance = p_bal, LastModified = SYSDATE
        WHERE CustomerID = p_id;
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Customer ' || p_id || ' updated.');
```

```
END;
```

```
    FUNCTION GetCustomerBalance(p_id IN NUMBER) RETURN NUMBER IS
        v_bal NUMBER;
    BEGIN
        SELECT Balance INTO v_bal FROM Customers
        WHERE CustomerID = p_id;
        RETURN v_bal;
    END;
```

```

END CustomerManagement;
/

-- test the package
BEGIN
    CustomerManagement.AddCustomer(4, 'Meera',
        TO_DATE('1992-11-05', 'YYYY-MM-DD'), 8000);
    CustomerManagement.UpdateCustomer(4, 'Meera R', 8500);
    DBMS_OUTPUT.PUT_LINE('Balance of customer 4: Rs.' ||
        CustomerManagement.GetCustomerBalance(4));
END;
/

```

Output:

Package created.

Package body created.

Customer Meera added.

Customer 4 updated.

Balance of customer 4: Rs.8500

PL/SQL procedure successfully completed.

Scenario 2: EmployeeManagement - package for employee operations

Program:

```

SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee(p_id    IN NUMBER,
                           p_name  IN VARCHAR2,
                           p_pos   IN VARCHAR2,
                           p_sal   IN NUMBER,
                           p_dept  IN VARCHAR2);

    PROCEDURE UpdateEmployee(p_id    IN NUMBER,
                             p_pos   IN VARCHAR2,
                             p_sal   IN NUMBER);

    FUNCTION GetAnnualSalary(p_id IN NUMBER) RETURN NUMBER;
END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee(p_id    IN NUMBER,
                           p_name  IN VARCHAR2,
                           p_pos   IN VARCHAR2,
                           p_sal   IN NUMBER,
                           p_dept  IN VARCHAR2) IS
    BEGIN
        INSERT INTO Employees (EmployeeID, Name, Position,

```

```

        Salary, Department, HireDate)
VALUES (p_id, p_name, p_pos, p_sal, p_dept, SYSDATE);
COMMIT;
DBMS_OUTPUT.PUT_LINE(p_name || ' hired as ' || p_pos || '.');
END;

PROCEDURE UpdateEmployee(p_id IN NUMBER,
                        p_pos IN VARCHAR2,
                        p_sal IN NUMBER) IS
BEGIN
    UPDATE Employees SET Position = p_pos, Salary = p_sal
    WHERE EmployeeID = p_id;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Employee ' || p_id || ' updated.');
```

END;

```

FUNCTION GetAnnualSalary(p_id IN NUMBER) RETURN NUMBER IS
    v_monthly NUMBER;
BEGIN
    -- assuming Salary column stores monthly salary
    SELECT Salary INTO v_monthly FROM Employees
    WHERE EmployeeID = p_id;
    RETURN v_monthly * 12;
END;
```

END EmployeeManagement;

/

```

BEGIN
    EmployeeManagement.HireEmployee(3, 'Ravi Kumar',
    'Analyst', 55000, 'Finance');
```

EmployeeManagement.UpdateEmployee(3, 'Senior Analyst', 65000);

```

    DBMS_OUTPUT.PUT_LINE('Annual salary of emp 3: Rs.' ||
    EmployeeManagement.GetAnnualSalary(3));
END;
```

/

Output:

Package created.

Package body created.

Ravi Kumar hired as Analyst.
Employee 3 updated.
Annual salary of emp 3: Rs.780000

PL/SQL procedure successfully completed.

Scenario 3: AccountOperations - package for account operations

Program:

```
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PACKAGE AccountOperations AS
```

```

PROCEDURE OpenAccount(p_acc_id IN NUMBER,
                     p_cust_id IN NUMBER,
                     p_type     IN VARCHAR2,
                     p_bal      IN NUMBER);

PROCEDURE CloseAccount(p_acc_id IN NUMBER);

FUNCTION GetTotalBalance(p_cust_id IN NUMBER) RETURN NUMBER;
END AccountOperations;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

PROCEDURE OpenAccount(p_acc_id IN NUMBER,
                     p_cust_id IN NUMBER,
                     p_type     IN VARCHAR2,
                     p_bal      IN NUMBER) IS
BEGIN
    INSERT INTO Accounts (AccountID, CustomerID, AccountType,
                          Balance, LastModified)
    VALUES (p_acc_id, p_cust_id, p_type, p_bal, SYSDATE);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Account ' || p_acc_id ||
                          ' opened for customer ' || p_cust_id);
END;

PROCEDURE CloseAccount(p_acc_id IN NUMBER) IS
BEGIN
    DELETE FROM Accounts WHERE AccountID = p_acc_id;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Account ' || p_acc_id || ' closed.');
```

```

END;

FUNCTION GetTotalBalance(p_cust_id IN NUMBER) RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT NVL(SUM(Balance), 0) INTO v_total FROM Accounts
    WHERE CustomerID = p_cust_id;
    RETURN v_total;
END;

END AccountOperations;
/

BEGIN
    AccountOperations.OpenAccount(3, 1, 'Savings', 5000);
    AccountOperations.OpenAccount(4, 1, 'Checking', 3000);
    DBMS_OUTPUT.PUT_LINE('Total balance of customer 1: Rs.' ||
        AccountOperations.GetTotalBalance(1));
    AccountOperations.CloseAccount(4);
END;
/
```

Output:

Package created.

Package body created.

Account 3 opened for customer 1
Account 4 opened for customer 1
Total balance of customer 1: Rs.9000
Account 4 closed.

PL/SQL procedure successfully completed.